

Credit Card Fraud Detection System

A Machine Learning Project Report

AUC-ROC: 0.9800

XGBoost · SMOTE · Streamlit

0.9800

AUC-ROC

87.76%

Recall

0.7345

F1 (tuned)

0.7406

MCC

284,807 rows

Dataset

0.17%

Fraud rate

Introduction

Credit card fraud causes billions in annual losses globally. Automated machine learning detection is essential for real-time screening at scale. This project builds a complete end-to-end fraud detection pipeline — from raw data exploration to a deployed interactive web application — on the Kaggle Credit Card Fraud dataset.

Abstract

Dataset: 284,807 European cardholder transactions (Sept 2013); 492 frauds (0.17%). Features: Time, Amount, V1–V28 (PCA-transformed). Target: Class (0/1). Pipeline: StandardScaler → stratified 80/20 split → SMOTE (train only) → Logistic Regression, Random Forest, XGBoost → AUC-ROC / Recall / F1 / MCC evaluation → threshold tuning → Streamlit deployment. **Best: XGBoost, AUC-ROC 0.9800, 86/98 fraud cases caught.**

Dataset Overview

Property	Detail
Source	Kaggle — ULB Machine Learning Group
Rows / Columns	284,807 rows × 31 columns
Fraud (Class=1)	492 (0.17% — 578:1 imbalance)
Legitimate (Class=0)	284,315 (99.83%)
Features	V1–V28 (PCA) + Amount + Time
Missing values	None — clean dataset

Tools Used

Tool / Library	Purpose
Python 3.9	Core language
Pandas & NumPy	Data manipulation & EDA
Matplotlib & Seaborn	Charts and visualizations
Scikit-learn	Preprocessing, models, metrics
XGBoost	Best-performing classifier
imbalanced-learn	SMOTE oversampling
Joblib	Model serialization (.pkl)
Streamlit	Interactive web dashboard

Steps Involved — A to Z

Step 1

Exploratory Data Analysis (EDA)

Loaded and profiled 284,807 rows — zero missing values confirmed. Plotted class distribution (bar and pie) revealing 578:1 imbalance. Analyzed Amount and Time distributions for fraud vs legitimate separately. Generated correlation heatmap and bar chart of each feature against the target. Identified V14, V12, V10, V17 as strongest fraud signals. Visualized top-9 feature distributions and violin plots for top-6 features.

Step 2

Data Preprocessing & SMOTE

Applied StandardScaler to Amount and Time only (V1–V28 already PCA-scaled). Performed 80/20 stratified train-test split (stratify=y) — 227,845 train, 56,962 test. Applied SMOTE to training set only (preventing data leakage): balanced from 394 to 227,451 fraud samples (1:1 ratio, total 454,902 training rows). Saved StandardScaler as scaler.pkl for consistent deployment scaling.

Step 3

Model Training

Trained three classifiers on SMOTE-balanced data: (1) Logistic Regression — class_weight='balanced', C=0.01, max_iter=1000; (2) Random Forest — 100 trees, max_depth=15, min_samples_leaf=5, n_jobs=-1; (3) XGBoost — 100 rounds, max_depth=5, learning_rate=0.1, subsample=0.8, colsample_bytree=0.8. All models saved as .pkl files.

Step 4

Model Evaluation

Evaluated on raw test set (56,962 rows, 98 fraud). Generated annotated confusion matrices, ROC curves, Precision-Recall curves, and a metrics heatmap. Computed AUC-ROC, Recall, Precision, F1, MCC, and Balanced Accuracy. Business cost analysis: \$150 per missed fraud (FN), \$5 per false alarm (FP). XGBoost threshold tuned from 0.50 to 0.85 — F1 rose from 0.40 to 0.73, false alarms reduced from 243 to 45. 5-fold CV on LR: AUC = 0.9825 ± 0.0099.

Step 5

Streamlit Dashboard

Built 4-tab web app: (1) Prediction tab — real-time fraud gauge, probability bar, risk tier badge, 3 input modes (sliders / paste / sample); (2) Analytics tab — feature importance chart, model comparison bar, confusion matrix; (3) Batch Predict tab — upload CSV, flag all fraud rows, download results; (4) About tab — full pipeline summary. Auto-trains from CSV if .pkl files absent. Deployable free on Streamlit Cloud via GitHub.

Model Evaluation Results — Test Set (56,962 transactions · 98 fraud cases)

Model	AUC-ROC	Recall	Precision	F1-Score	MCC	Fraud Caught	Fraud Missed
Logistic Regression	0.9700	0.9184	0.0581	0.1094	0.1060	90 / 98	8
Random Forest	0.9788	0.8776	0.2976	0.4444	0.4200	86 / 98	12
XGBoost (default 0.50)	0.9800	0.8776	0.2614	0.4028	0.4776	86 / 98	12
XGBoost (tuned 0.85)	0.9800	0.8469	0.6486	0.7345	0.7406	83 / 98	15

Selected model: XGBoost (tuned threshold 0.85) — highest F1 (0.7345) and MCC (0.7406). **Primary metric is Recall** — missing fraud costs more than a false alarm in any real-world system.

Key Findings & Observations

Class Imbalance is Severe

Only 492 of 284,807 transactions are fraudulent (0.17%, ratio 578:1). Plain accuracy scores 99.83% while catching zero fraud — Recall, AUC-ROC, and MCC are the correct metrics for this problem.

SMOTE Balances Training Data

SMOTE applied only to training set created 226,957 synthetic fraud samples, achieving a 1:1 class ratio (227,451 each). PCA scatter confirmed synthetic samples cluster naturally near real fraud cases — no data leakage.

V14 Dominates Predictions

XGBoost feature importance: V14 = 52.4% of model decisions. V4 (4.8%), V12 (3.8%), V8 (3.6%) follow. Transactions with V14 below -5 consistently carry very high fraud risk.

Threshold Tuning Dramatically Improves F1

Default threshold 0.50 → F1=0.40. Tuned threshold 0.85 → F1=0.73, MCC=0.74. False alarms reduced from 243 to 45 while recall remains above 84%.

MCC is Most Reliable Metric

Matthews Correlation Coefficient accounts for all four confusion matrix cells. Unlike F1, it correctly handles the case where true negatives vastly outnumber positives — the most trustworthy metric at 578:1 imbalance.

Plots & Charts Generated

Chart	Description
Class distribution	Bar + pie showing 578:1 imbalance
Amount analysis (3 plots)	Fraud vs legit amount comparison & boxplot
Time analysis	Fraud frequency by hour of day
Feature correlation bar	Each feature's Pearson correlation with Class
Correlation heatmap	Top feature inter-correlations
Top-9 feature distributions	Density plots — fraud vs legitimate
Violin plots	Spread of top-6 features by class
Before/after scaling	StandardScaler effect on Amount & Time
SMOTE comparison	Class balance before and after SMOTE
SMOTE PCA scatter	Real vs synthetic fraud sample placement
Confusion matrices	TP/TN/FP/FN for all 3 models
ROC curves	AUC-ROC comparison across all models
Precision-Recall curves	PR curve — better metric for imbalance
Metrics heatmap	All 7 metrics for all 3 models
Business cost chart	Financial impact of FN vs FP per model
Threshold tuning	F1 / Recall / Precision vs threshold sweep
Feature importance	Top 15 features — Random Forest & XGBoost

Conclusion

This project successfully demonstrates a complete, production-ready machine learning pipeline for credit card fraud detection, covering every stage of the ML lifecycle from raw data exploration through model deployment.

The XGBoost classifier trained on SMOTE-balanced data achieved an AUC-ROC of **0.9800**, correctly identifying **86 of 98 fraud cases** on the unseen test set. Threshold tuning improved F1 from 0.40 to **0.7345** and MCC from 0.48 to **0.7406**, reducing false alarms from 243 to 45 while maintaining 84.7% recall.

Four critical lessons were established: (1) plain accuracy is misleading for imbalanced data — Recall and MCC are the correct metrics; (2) SMOTE must be applied only after the train-test split to prevent data leakage; (3) MCC is the most reliable single metric when class sizes differ drastically; (4) decision threshold tuning is essential in production fraud systems where the cost of missed fraud far exceeds the cost of a false alarm.

The Streamlit dashboard makes the model accessible to non-technical users, supporting live single-transaction scoring, batch CSV prediction, and an interactive model analytics panel.

Streamlit Dashboard

Tab	Functionality
Prediction	Real-time fraud gauge, probability bar, risk tier badge, 3 input modes: slider
Analytics	Feature importance chart, model comparison bar chart, confusion matrix wit
Batch Predict	Upload any CSV → predict all rows → view flagged transactions → downloa
About	Full pipeline overview and model explanation

Project Deliverables

File	Description
Phase1_EDA.ipynb	EDA notebook — 8 plots
Phase2_Preprocessing.ipynb	Scaling, split & SMOTE — 3 plots
Phase3_Training.ipynb	Model training — 5 plots
Phase4_Evaluation.ipynb	Full evaluation — 8 plots
app.py	Streamlit app (4 tabs, 775 lines)
requirements.txt	Python dependencies
model_xgboost_best.pkl	Saved XGBoost model
scaler.pkl	Fitted StandardScaler
best_threshold.pkl	Optimal threshold (0.85)